

Base Rules

1. No manual coding, all code / files to be generated with AI Tool
2. Tools (Cursor - As everyone already have it)
3. Develop as an actual project which you would do with full ownership and gradual progression

Development Timeline

The exercise is self-paced and meant to be completed within one week. You may work in any order; there is no required day-wise plan. Share your work by the agreed date.

Expected effort: the mandatory Core of the project is scoped for roughly 8 - 12 focused hours. The rest of the week goes into the lifecycle artifacts - requirement analysis, prompt history, testing and debugging notes, reflection - which are the main things the feedback looks at. Do not expand the Core application at the expense of these artifacts.

Github Repo submission: By 21st July

Assessment by Mentor : By 31st July

Common Technical Requirements

Every submission must include:

1. Frontend application (any JS library)
2. Backend API (any Python Framework)
3. Database persistence
4. Database setup or migration scripts
5. Seed or sample data
6. Input validation
7. Error handling
8. One working search or filter capability (Core); more in Stretch
9. At least one meaningful test tier (Core); more in Stretch
10. README setup instructions
11. Prompt history (export of selected Cursor chats - 10/15 sessions)
12. All planning, design, testing, debugging, review, reflection, and PR artifacts in the repository structure (in /artifacts folder)

Database Requirement

Use Any RDBMS. Provide your database choice, setup instructions, a schema/migration/initialization script, seed data, an environment variable example (if applicable), and steps to run locally.

Authentication

Authentication is optional. If implemented well it counts as Stretch evidence (login/logout, JWT or session auth, role-based access, protected routes, API authorization).

What You Get Out of It

You receive a feedback report: your strengths, your growth areas, and concrete next steps for developing your AI workflow. The report also gives a sense of where you currently sit in the AI capability framework and what would move you forward - think of it as a snapshot and a direction, not a grade. Your competency owner uses this as one input into your normal growth track.

Feedback focuses on areas like requirement analysis, prompting and context-setting, tool workflow, full-stack design, code quality, database design, testing depth, debugging, code review, documentation, ownership, and responsible AI judgment - so you leave with a clear picture of what to practice next.

5. How the Exercise is Structured

It has three parts:

Part	Focus	Emphasis
Part A	AI Workflow Foundation	20%
Part B	Full-Stack Mini Project (Core + optional Stretch)	60%
Part C	Submission and Reflection	20%

Part C is completed through the participation form (see “How to Take Part”), where you share your repository and answer a few questions about your work. The percentages show where to put your effort — they are not exam weights.

Part A: AI Workflow Foundation

Objective

Show that you understand how AI should be used in a practical engineering workflow — thoughtfully, not as a simple code-generation shortcut.

Expected Submission

Submit a document named `tool-workflow.md` covering:

1. Primary AI tool used.
2. How you provide project context to the tool.
3. How you use AI for requirement analysis.
4. How you use AI for planning and design.
5. How you use AI for code generation.
6. How you validate AI-generated code.
7. How you use AI for testing.
8. How you use AI for debugging.
9. How you use AI for code review.
10. What information you avoid sharing unnecessarily with AI tools.
11. How you would reuse this workflow in a real project.

Project

Support Ticket Management System

Business Context

A small application for managing support tickets. Internal users create, update, comment on, search, and progress tickets through a defined lifecycle.

Core (Mandatory)

Entities

User (seeded only — no user-management UI required)
- id, name, email, role

Ticket
- id, title, description, priority, status,
assignedTo, createdBy, createdAt, updatedAt

Comment
- id, ticketId, message, createdBy, createdAt

Features

1. Create a ticket.
2. List tickets.
3. View ticket details.
4. Update ticket fields (title, description, priority, assignee).
5. Change ticket status through the enforced state machine.
6. Add comments to a ticket.
7. Persist all data; data survives restart.
8. Validate required fields; reject invalid input at the backend.
9. Show meaningful error states in the UI.
10. User can export all self-generated tickets with details as csv.

Status state machine

This is the signature judgment area.

Allowed transitions:

- Open -> In Progress
- In Progress -> Resolved
- Resolved -> Closed
- Open -> Cancelled
- In Progress -> Cancelled

Invalid transitions must be rejected by the backend and handled clearly in the frontend.

Mandatory test requirement

Integration tests must prove the state-machine rules:

- valid transitions succeed
- invalid transitions are rejected

Stretch (Optional — evidence toward C1.1)

- Richer data model
- Full user CRUD with basic user roles
- Authentication, protected routes, API authorization checks
- API documentation (Swagger / OpenAPI)
- Docker setup
- Reusable prompt templates, rules, or specs

Core Acceptance Criteria

1. A user can create a ticket via the UI.
2. A user can view all tickets from the database.

3. A user can open a ticket detail view.
4. A user can update ticket fields and reassign.
5. A user can add comments.
6. Status changes only through valid transitions; invalid ones are rejected.
7. Data remains available after restart.
8. Backend validation prevents invalid records.
9. No secrets committed to the repo.
10. User can export all self-generated tickets with details as csv
11. State-machine integration tests pass.

What Counts as Complete

For your feedback to be useful, your submission should include a working frontend, working backend, database persistence, database setup or migration files, seed data, README setup instructions, basic tests, prompt history, requirement analysis, design notes, reflection, and a PR description. If pieces are missing you will still get feedback — it just won't reflect the full picture, and the growth pointers will focus on filling those gaps.

How to Take Part

Once your project is ready, you share it through a single online form. There is no review call to book and nothing to host or deploy.

What you share

1. A link to your Git repository (make sure it is accessible to the competency team).
2. Your selected project option and primary AI tool.
3. Short written answers to a few questions in the form.

About the questions

The questions ask you to explain your work in your own words — your requirement understanding, how you used AI across the lifecycle, key code and design decisions, your testing and debugging approach, and what you'd improve. A few ask you to point to specific places in your repository (for example, the commit where you fixed an AI mistake), so keep your commit history and prompt history organized as you go.

Be specific and honest. This is for your own development, so there's no value in generic or inflated answers — they just produce less useful feedback. The clearer the picture you give of how you actually worked, the more useful your growth pointers will be.

Follow-up and coaching

Your work is reviewed and a feedback report is shared with you. Sometimes a mentor or your competency owner may follow up to talk through your project and coach you on next steps — a short, supportive conversation, not a re-examination. This is part of how the competency helps you grow, not a hurdle.

What Good Looks Like

The feedback weighs how you used AI across the lifecycle and how well you understand and own your solution — not just whether the app runs. At a high level:

Strong work usually shows

- Clear understanding of the problem before coding, with good requirement breakdown and acceptance criteria
- Well-documented AI prompts with iteration — context-setting, refinement, and correction of wrong suggestions
- Clean, maintainable code and a working database setup that runs from the README
- Meaningful tests and clear evidence of debugging and review
- An honest reflection, the ability to explain trade-offs, and reusable workflow artifacts

Weaker work usually shows

- Direct copy-paste from AI with little understanding; missing requirement analysis
- Missing or shallow prompt history; no clear design
- Broken setup instructions or no database persistence

- Superficial tests; no debugging or review evidence
- Generic documentation; inability to explain the generated code; no ownership of the result

In short: the journey and your understanding matter as much as the final code. Make your thinking visible.

Your Growth Path

Whatever your starting point, the feedback comes with a direction for what to develop next:

- **Building the basics:** focus on requirement analysis, providing context to your AI tool, and getting to working, tested output. A follow-up exercise or learning path may be suggested.
- **Solid and growing:** you produce working AI-assisted output; next is deepening tests, review quality, and debugging maturity.
- **Strong across the lifecycle:** you use AI well end-to-end and can explain your decisions; next is advanced testing and reusable workflows.
- **Ready to lead:** mature workflow with reusable prompts, rules, or specs — a natural point to start sharing practices and mentoring others.

Your competency owner uses the report and this direction within your normal growth conversations — it informs where you are in the capability framework rather than stamping a grade on you.

Summary

You'll complete a self-paced, one-week AI-assisted full-stack exercise using an approved AI tool (Cursor). Choose one project option — Backend-heavy (Support Ticket Management System) — and build the mandatory Core; Stretch is optional and shows more advanced practice.

Your project should include a frontend, a backend API, database persistence, setup/migration files, seed data, validation, error handling, search/filter, tests, README instructions, AI prompt/workflow history, a reflection, and a PR description. Authentication is optional.

Use AI thoughtfully and make your usage visible — how you set context, planned, generated and refined code, tested, debugged, reviewed, and documented. When you're ready, share your repository link and answers through the form. You'll get a feedback report with your strengths, growth areas, and next steps, and possibly a short coaching conversation. Everyone in the competency does this — it's a shared step in building how we all work with AI.

Required Repository Structure

Submit a Git repository following this structure as closely as possible:

```
ai-practical-assessment/  
  README.md  
  candidate-info.md  
  tool-workflow.md  
  requirements-analysis.md  
  acceptance-criteria.md  
  implementation-plan.md
```

```
design-notes.md
api-contract.md  data-model.md  ui-flow.md
test-strategy.md
src/  tests/
database/
  schema-or-migrations  seed-data  setup-notes.md
test-results.md  debugging-notes.md
code-review-notes.md  review-fixes.md
pr-description.md  reflection.md  final-ai-usage-summary.md
ai-prompts/
  planning.md  design.md  implementation.md
  testing.md  debugging.md  code-review.md  documentation.md
tool-specific/
  cursor-workflow/
```

Fill only the tool-specific folder relevant to your selected tool.

Submission Templates

Use these as starting structures for the required artifacts — they are a floor, not a limit.

Candidate Information

File: candidate-info.md

```
# Candidate Information
Name: / Role: / Primary Technology Stack:
Primary AI Tool Used: / Project Option Selected:
Assessment Start Date: / Submission Date:
## Project Summary / ## Tools Used / ## Setup Summary
```

Requirement Analysis

File: requirements-analysis.md

```
# Requirement Analysis
## Selected Project Option
## My Understanding (in your own words)
## Functional Requirements / ## Non-Functional Requirements
## Assumptions / ## Clarifications (questions for a product owner)
## Edge Cases
```

Acceptance Criteria

File: acceptance-criteria.md

```
# Acceptance Criteria
## Core - [ ] ... ## Validation - [ ] ...
## Error Handling - [ ] ... ## Testing - [ ] ...
## Documentation - [ ] ...
```

Implementation Plan

File: implementation-plan.md

```
# Implementation Plan
## Overview / ## Task Breakdown / ## Milestones
## AI Usage Plan / ## Risks / ## Mitigation
```

Design Notes

File: design-notes.md

```
# Design Notes
## Architecture Overview (frontend, backend, database)
## Frontend Design / ## Backend Design / ## Database Design
## Validation Strategy / ## Error Handling Strategy
## Testing Strategy Link
```

API Contract

File: api-contract.md

```
# API Contract
## Endpoint: Method: Path: Purpose:
### Request { } ### Response { }
### Validation Rules ### Error Responses
```

Test Strategy

File: test-strategy.md

```
# Test Strategy
## Test Scope
## Unit Tests / ## Component Tests
## API / Integration Tests / ## Edge Case Tests
## Tests Not Covered (and why)
```

Debugging Notes

File: debugging-notes.md

```
# Debugging Notes
## Issue 1
### Problem / ### How I Investigated
### How AI Helped / ### What I Validated / ### Final Fix
```

Code Review Notes

File: code-review-notes.md

```
# Code Review Notes
## AI-Assisted Review Summary / ## My Review Observations
## Changes Made After Review / ## Suggestions Rejected (and why)
```

Reflection

File: reflection.md

```
# Reflection
## What I Built / ## How I Used AI (across the lifecycle)
## What AI Helped With Most / ## What AI Got Wrong
## How I Validated AI Output / ## What I Would Improve Next
## Reusable Workflow (prompts, rules, specs, templates)
```

PR Description

File: pr-description.md

```
# PR Description
## Summary / ## Features Implemented / ## Technical Changes
## Database Changes / ## Testing Done / ## AI Usage Summary
## Screenshots / Demo Notes / ## Known Limitations / ## Future Improvements
```

Prompt History Expectations

Group prompt history by activity under `ai-prompts/`. It need not include every interaction, but it must show enough to evaluate how you used AI. For each prompt, capture: the prompt text or summary, the AI response summary, what you accepted, what you changed, what you rejected, and why.

Strong prompt history shows: context setting, iteration, review of AI output, correction of wrong suggestions, stack-specific guidance, testing and validation prompts, and responsible judgment about what context to share.

Weak prompt history shows: one-line prompts only, no context, no iteration, no evidence of review, only final answers copied from AI, and missing prompts for testing, debugging, or review.